

INSTITUTO BRASILEIRO DE DESENVOLVIMENTO E PESQUISA

Ciência da Computação · Engenharia de Software

WORKSHOP DE DOCKER

*Aula 1: Fundamentos teóricos, instalação
e primeiros containers*

Samuel Abrão

Brasília · 2026

Sumário da aula

- 1. Apresentação do workshop
- 2. Fundamentos de sistema operacional: kernel, processo, daemon
- 3. O que é Docker e como funciona
- 4. Imagens
- 5. Instalação assistida
- 6. Primeiros comandos na CLI

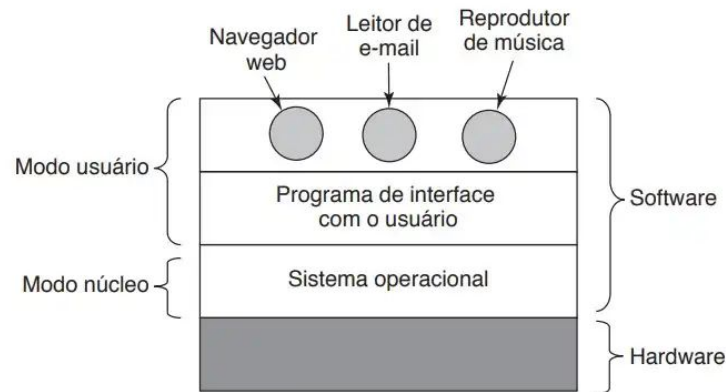
PARTE 1

1

Fundamentos de Sistema Operacional

O que é um sistema operacional

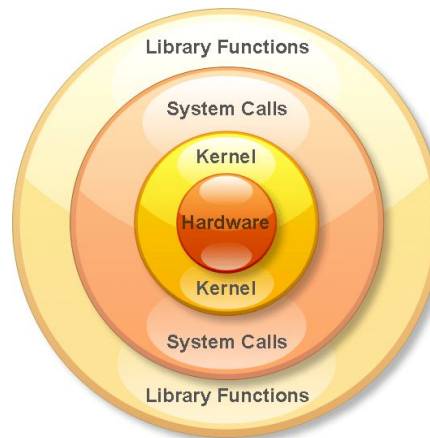
- Programa que executa em modo privilegiado
- Camada intermediária entre hardware e programas de usuário
- Duas funções: oferecer abstração do hardware e gerenciar recursos
- É essa segunda função que torna o Docker possível



TANENBAUM; BOS (2016, p. 1-2).

Kernel e modos de execução

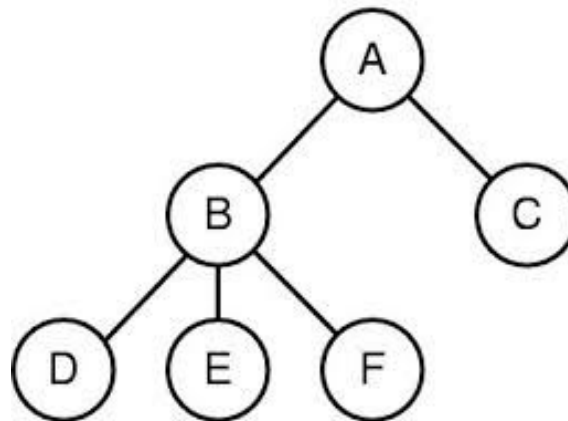
- O Kernel executa em modo privilegiado (modo kernel ou supervisor)
- Todo programa "normal" executa em modo usuário, que é restrito
- Comunicação entre os modos: Syscalls (chamadas de sistema)
- Docker pede pro Kernel isolar



TANENBAUM; BOS (2016, p. 4-5).

Processos

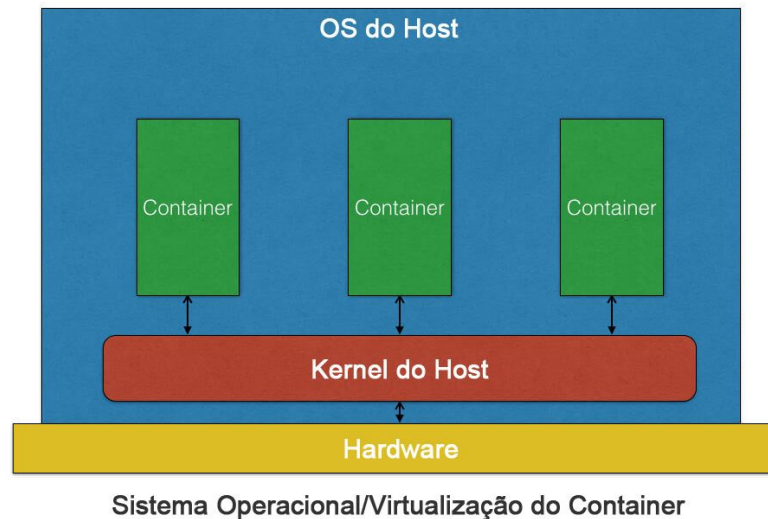
- Processo é uma instância de um programa em execução
- Cada processo tem PID, memória própria, arquivos abertos, sinais
- Processos formam uma árvore. PID 1 é o ancestral de todos (init/systemd)
- Isolamento entre processos já existe no SO, não é coisa do Docker



TANENBAUM; BOS (2016, p. 85-86).

Processos de segundo plano

- Processo de longa duração executando em background
- Sem terminal associado, provê serviços a outros processos
- Convenção UNIX: nomes terminam em "d" (sshd, crond, systemd)
- O Docker é um daemon: dockerd



TANENBAUM; BOS (2016, p. 730-731).

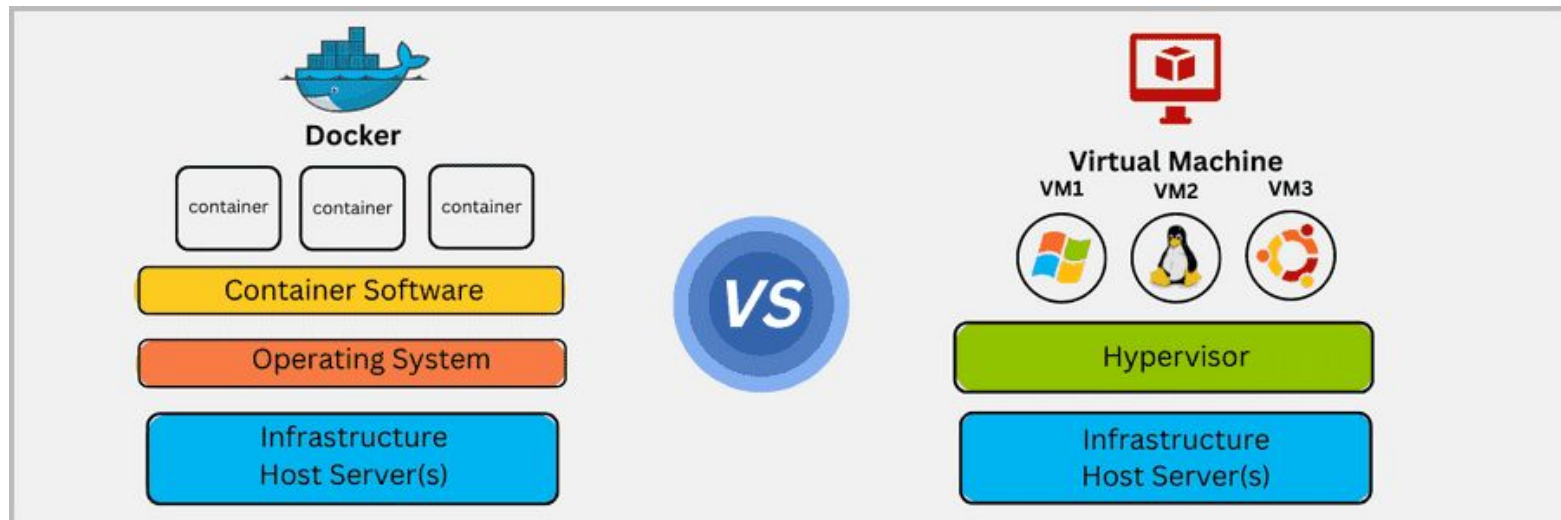
PARTE 2

2

O que é o Docker

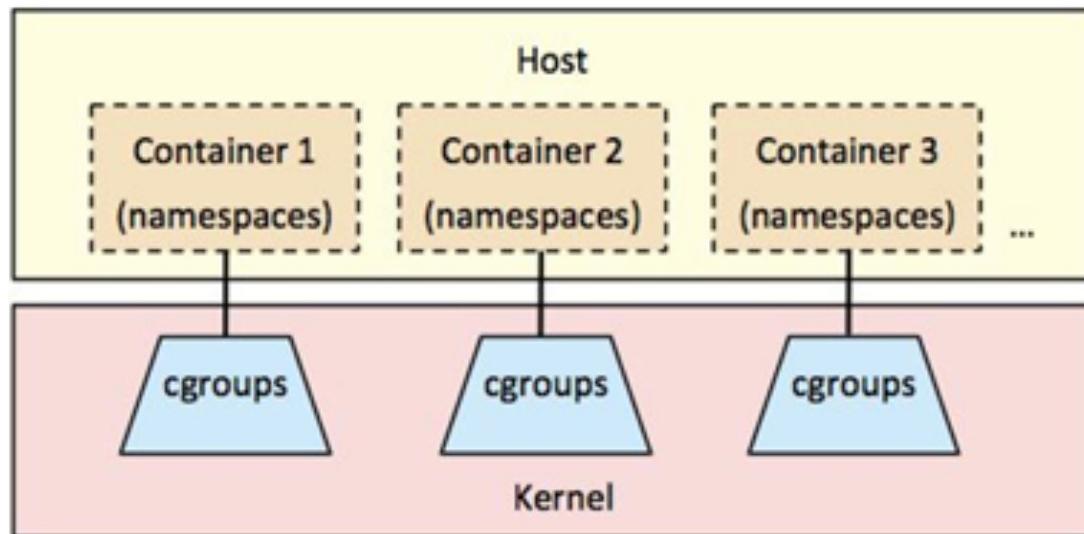
Duas abordagens de isolamento

- Máquina virtual: simula hardware completo e roda outro SO
- Container: usa o kernel do host, processo isolado por namespaces
- 10 VMs significam 10 kernels rodando. 10 containers, 1 kernel só
- Resultado: containers consomem muito menos memória e disco



Como o Docker funciona por baixo dos panos

- Duas funcionalidades do kernel Linux: namespaces e cgroups
- Namespaces isolam o que o processo enxerga (PIDs, rede, filesystem)
- Cgroups limitam o que o processo consome (CPU, memória, I/O)
- Docker não isola. O kernel isola. Docker só pede.



LINUX (2024). namespaces(7), Linux manual page.

Modelo cliente-servidor do Docker

- Cliente: comando docker no terminal
- Servidor: daemon dockerd em background
- Comunicação via socket: /var/run/docker.sock
- Cadeia: docker → dockerd → containerd → runc → kernel
- Quem controla o socket controla a máquina (atenção a permissões)

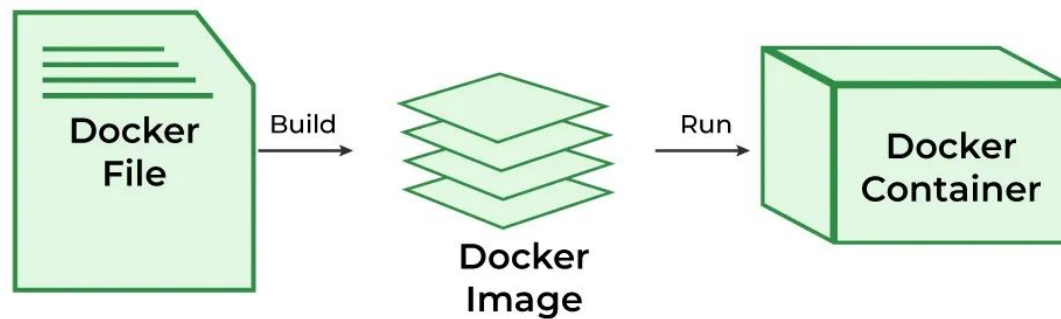
PARTE 3

3

Imagens

O que é uma imagem Docker

- Imagem: template imutável com filesystem e metadados
- Container: imagem em execução (instância)
- Imagem não muda. Containers ganham uma camada de escrita por cima.



Imagens são organizadas em layers

- Cada instrução em um Dockerfile gera uma camada
- Camadas comuns são compartilhadas entre imagens (economia de disco)
- pull baixa apenas as camadas que ainda não existem localmente
- build reaproveita cache de camadas, então rebuilds são rápidos

```
FROM node:22-alpine          # Imagem baseada em NodeJS utilizando Linux Alpine

WORKDIR /app                 # Define o diretório de trabalho do container

COPY package*.json ./        # Copia os arquivos e dependências

RUN npm install              # Instala deps do projeto (Pesado → pode ser cacheado)

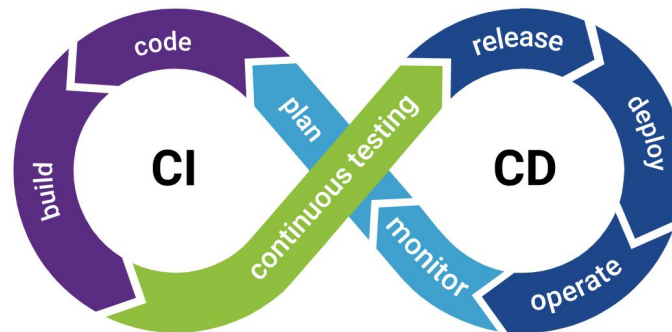
COPY . .                     # Copia todos os arquivos do programa

EXPOSE 3000                  # Expõe a porta 3000 da rede pra acesso externo

CMD ["npm", "run", "dev"]     # Comando para iniciar container (pode ser um script)
```

Imagens no fluxo de desenvolvimento

- Parte-se sempre de uma imagem oficial (python, node, postgres)
- Imagens vivem em registries: Docker Hub, ECR, GHCR, Artifact Registry
- Empresas usam registries privados (Harbor, ECR) para imagens internas
- Fluxo típico: git push, CI builda imagem, push pro registry, deploy
- A unidade de deploy moderna não é mais o código. É a imagem.



PARTE 4

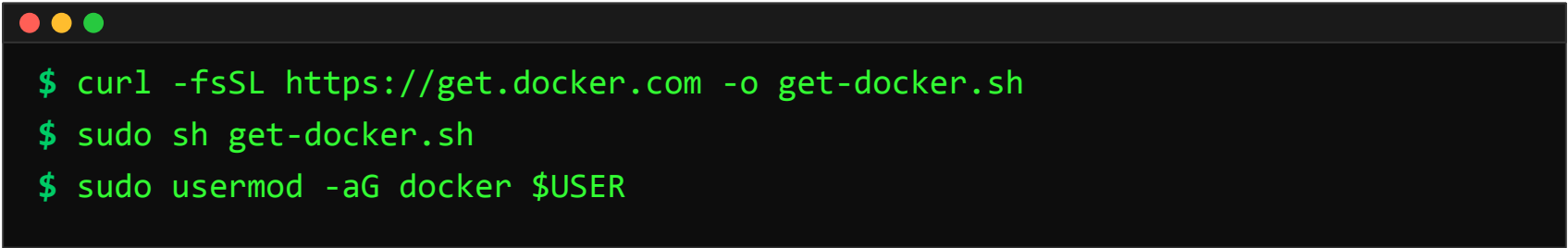
4

Instalação assistida



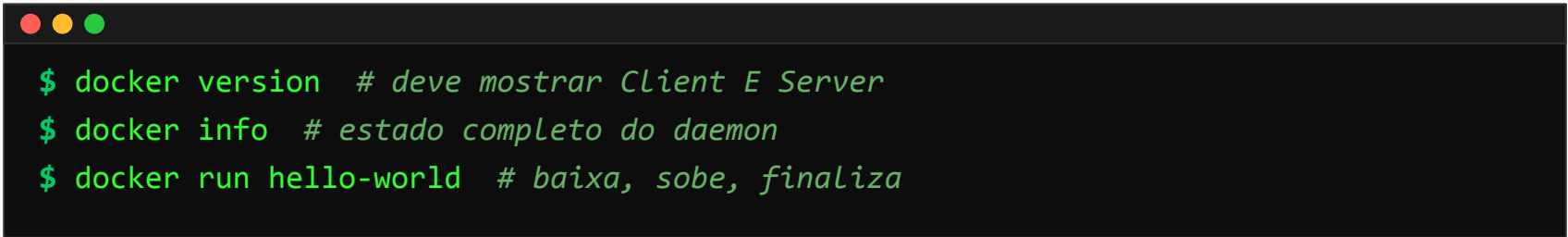
Instalação em Ubuntu, Fedora e derivados

- Script oficial de conveniência da Docker
- Detecta a distribuição e configura repositórios automaticamente
- Após instalar: adicionar o usuário ao grupo docker (sem sudo)
- Fechar e abrir o terminal para o novo grupo passar a valer

A terminal window with a dark background and green text. It shows three commands being executed in sequence. The first command is `$ curl -fsSL https://get.docker.com -o get-docker.sh`. The second command is `$ sudo sh get-docker.sh`. The third command is `$ sudo usermod -aG docker $USER`. The terminal window has three colored window control buttons (red, yellow, green) in the top left corner.

```
$ curl -fsSL https://get.docker.com -o get-docker.sh
$ sudo sh get-docker.sh
$ sudo usermod -aG docker $USER
```

Confirmar que o Docker está rodando

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains three lines of text in a green monospace font, each preceded by a dollar sign (\$).

```
$ docker version # deve mostrar Client E Server  
$ docker info # estado completo do daemon  
$ docker run hello-world # baixa, sobe, finaliza
```

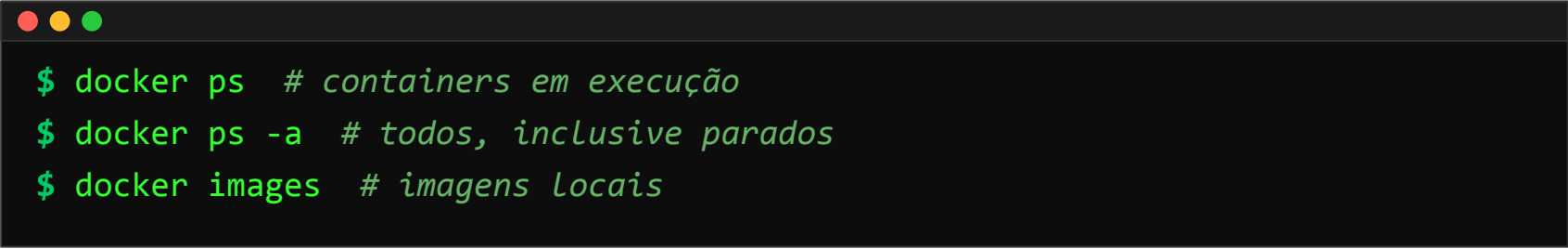
- Se aparecer só Client em docker version, o daemon não está rodando
- Linux: `sudo systemctl start docker`
- Windows/macOS: abrir o Docker Desktop
- Plano B se travar: labs.play-with-docker.com (roda no navegador)

PARTE 5

5

Primeiros comandos na CLI

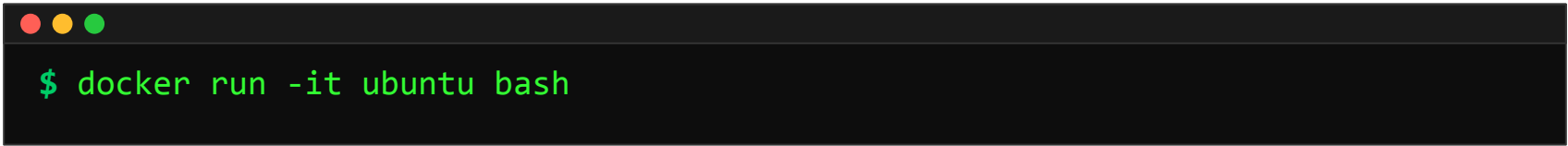
Inspecionar o estado atual



```
$ docker ps # containers em execução  
$ docker ps -a # todos, inclusive parados  
$ docker images # imagens locais
```

- `docker ps` mostra apenas containers em execução naquele momento
- `docker ps -a` mostra também os parados (úteis para inspecionar logs)
- `docker images` lista as imagens já baixadas para sua máquina

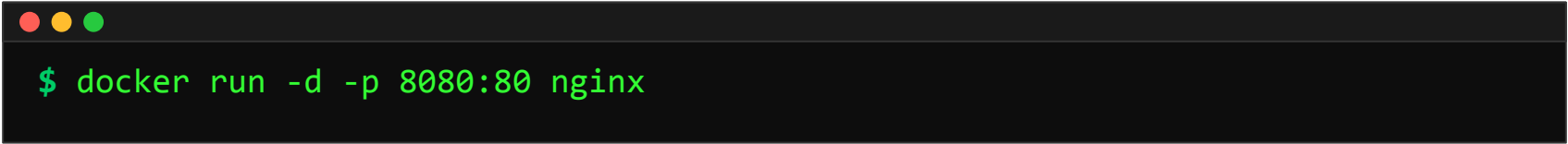
Entrar em um container como num shell comum

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text '\$ docker run -it ubuntu bash' is displayed in a light green monospace font.

```
$ docker run -it ubuntu bash
```

- -i = interativo, -t = TTY (terminal)
- Caem dentro de um shell rodando no container
- Testar: ls, ps, cat /etc/os-release
- É outro filesystem, outros processos visíveis, mas o mesmo kernel
- Digitar exit faz o bash terminar, e o container termina junto

Subir um serviço acessível pelo navegador



```
$ docker run -d -p 8080:80 nginx
```

- -d = detached (executa em background)
- -p 8080:80 = mapeia porta 8080 do host para porta 80 do container
- Acessar <http://localhost:8080> no navegador
- docker stop faz o nginx sumir, e o sistema do host continua limpo
-

Encerrar e remover



```
$ docker stop <id> # para o container
$ docker rm <id> # remove container parado
$ docker rmi <imagem> # remove a imagem do disco
```

- `docker stop` envia SIGTERM ao processo principal do container
- `docker rm` libera o nome e o espaço em disco do container
- `docker rmi` remove a imagem (só funciona se nenhum container a usa)
- Com o uso, esses comandos se tornam naturais. Não precisa decorar agora.

O que vimos hoje

- Sistema operacional: kernel privilegiado, processos e daemons
- Container é diferente de VM: usa o kernel do host via namespaces e cgroups
- Docker: arquitetura cliente-servidor (docker → dockerd → containerd → runc)
- Imagem: template imutável em camadas, unidade de deploy moderna
- Instalação concluída e primeiros comandos executados

Bibliografia da aula

- TANENBAUM, A. S.; BOS, H. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson Education do Brasil, 2016.
- DOCKER. What is a container? Docker Documentation, 2024. Disponível em: docker.com/resources/what-container. Acesso em: 16 maio 2026.
- DOCKER. Docker overview: Docker architecture. Docker Documentation, 2024. Disponível em: docs.docker.com/get-started/docker-overview. Acesso em: 16 maio 2026.
- LINUX. namespaces(7), Linux manual page. The Linux Kernel Archives, 2024. Disponível em: man7.org/linux/man-pages/man7/namespaces.7.html. Acesso em: 16 maio 2026.